

LABORATÓRIO 05 - GraphQL vs REST - Um experimento controlado

1. Introdução e Desenho do Experimento

O desenvolvimento de sistemas modernos baseados em microserviços e arquiteturas open-source exige a escolha de modelos de comunicação eficientes e escaláveis. Historicamente, as APIs REST (Representational State Transfer) têm sido o padrão de mercado. No entanto, sua rigidez estrutural baseada em endpoints fixos frequentemente resulta em dois problemas clássicos de transferência de dados: o *over-fetching* (transferência de dados sobressalentes desnecessários) e o *under-fetching* (necessidade de realizar múltiplas requisições sequenciais para consolidar um único objeto complexo de negócio, também conhecido como o problema N+1).

Como alternativa, a especificação GraphQL propõe um modelo declarativo baseado em grafos, permitindo que o cliente molde a resposta de acordo com suas necessidades específicas de consumo. Este relatório documenta um experimento controlado empírico cujo objetivo é avaliar quantitativamente as diferenças de desempenho de rede e volumetria entre as duas arquiteturas utilizando como objeto experimental a infraestrutura oficial do GitHub (API v3 REST e API v4 GraphQL).

Perguntas de Pesquisa (Research Questions)

- **RQ1.** Respostas às consultas GraphQL são mais rápidas que respostas às consultas REST?
- **RQ2.** Respostas às consultas GraphQL têm tamanho menor que respostas às consultas REST?

A. Hipóteses Nulas (H0) e Alternativas (H1)

- **H0 (Tempo - RQ1):** Não há diferença estatisticamente significativa no tempo de resposta acumulado entre as rotinas de consultas REST e GraphQL na API do GitHub para a extração do mesmo volume de dados de interesse.
- **H1 (Tempo - RQ1):** O tempo de resposta acumulado das rotinas de consultas GraphQL é significativamente menor do que o das consultas REST na API do GitHub devido à resolução centralizada de nós aninhados.
- **H0 (Tamanho - RQ2):** Não há diferença estatisticamente significativa no tamanho do payload (tráfego total de bytes) de resposta entre as consultas REST e GraphQL na API do GitHub.

- **H1 (Tamanho - RQ2):** O tamanho total do payload de resposta das consultas GraphQL é significativamente menor do que o das consultas REST na API do GitHub como consequência da eliminação de campos ociosos.

B. Definição das Variáveis

- **Variáveis Dependentes:**
 - **Tempo de Resposta:** Cronometrado em segundos, compreendendo o intervalo decorrido entre o disparo do método de requisição HTTP pelo cliente até o encerramento do recebimento do fluxo de dados pelo servidor.
 - **Tamanho da Resposta:** Medido de forma objetiva em bytes com base no tamanho do corpo bruto do conteúdo textual criptografado ou em formato JSON retornado (`len(response.content)`).
- **Variável Independente:** O estilo arquitetural de comunicação Web empregado, assumindo os níveis categóricos discretos: REST e GraphQL.

C. Tratamentos, Objetos e Amostragem

O cenário prático adota a extração de métricas reais de *Code Review* de repositórios populares. A meta consiste em minerar dados de Pull Requests com status fechado ou integrado (CLOSED ou MERGED), cuja duração de revisão seja estritamente superior a 1 hora e que possuam ao menos 1 revisão formal mapeada. A janela de amostragem visa cobrir os 100 repositórios públicos mais populares ordenados por volume de estrelas (Stars), inspecionando até 100 PRs por projeto.

D. Ameaças à Validade

- **Validade Interna:** Oscilações intermitentes na latência da rede local e variações de banda do provedor de internet durante a execução sequencial dos scripts. Além disso, o mecanismo reativo de proteção contra abusos de requisições do GitHub (*Rate Limiting*) penaliza rotinas massivas e sequenciais, introduzindo tempos de espera artificiais necessários para mitigar erros HTTP 403 e 429.
- **Validade de Construto:** O tempo capturado no cliente final inclui não apenas o custo computacional de processamento e serialização interna das queries nos servidores do GitHub, mas também o tempo de trânsito físico dos pacotes TCP/IP na rede global.
- **Validade Externa:** Como o experimento avalia estritamente os barramentos públicos da plataforma GitHub, as conclusões empíricas geradas podem não representar de maneira generalista o comportamento de microserviços corporativos internos cuja otimização de banco de dados e políticas de infraestrutura difiram do ecossistema estudado.

2. Preparação do Experimento e Metodologia

A infraestrutura de teste foi implementada utilizando scripts automatizados em linguagem Python, tirando proveito da biblioteca requests para abstrair as camadas de transporte HTTP. As métricas foram consolidadas e geradas em arquivos estruturados utilizando a biblioteca pandas.

A fim de mitigar a assimetria na coleta de dados e garantir a equivalência lógica das transações, os dois modelos foram instrumentados para seguir estratégias específicas de controle de tráfego de dados:

1. **Rotina GraphQL:** O cliente dispara uma consulta aninhada profunda que solicita de imediato os dados do repositório, o nó de metadados dos Pull Requests e a contagem explícita de revisões de código de cada PR. Para evitar o estouro de limites de recursos de consultas complexas do servidor (RESOURCE_LIMITS_EXCEEDED), o tamanho de lote da paginação do loop foi calibrado estritamente para 5 repositórios por requisição.
2. **Rotina REST (Otimizada):** Em decorrência do isolamento de endpoints do modelo REST, o código foi otimizado aplicando a técnica de curto-circuito (*short-circuiting*). O script faz 1 chamada inicial para buscar repositórios e 1 chamada por repositório para baixar a lista de PRs. O cálculo de tempo de duração do PR é feito em memória local utilizando os dados nativos do JSON de pulls. O script realiza a requisição extra e aninhada de reviews (o ponto N+1 da arquitetura) unicamente se o Pull Request em questão for validado previamente no filtro temporal de duração maior que 1 hora, reduzindo o desperdício de chamadas de rede inúteis.

3. Execução do Experimento e Resultados Coletados

Ambos os tratamentos foram executados contra a infraestrutura de produção da API do GitHub. Os dados consolidados extraídos diretamente dos arquivos gerados pelos logs de instrumentação (metricas_desempenho_graphql.csv e metricas_desempenho_rest.csv) revelam disparidades brutais de desempenho e consumo.

Métrica de Infraestrutura	Tratamento REST (v3)	Tratamento GraphQL (v4)	Fator de Impacto / Diferença
Volume Total de Requisições HTTP	4.292 chamadas	22 chamadas	REST fez 195x mais reqs
Tempo Acumulado de Execução (RQ1)	1.838,02 segundos (~30,63 min)	87,81 segundos (~1,46 min)	GraphQL foi 21x mais rápido
Volume de Dados	182.043.675 bytes	1.623.729 bytes	REST consumiu

Métrica de Infraestrutura	Tratamento REST (v3)	Tratamento GraphQL (v4)	Fator de Impacto / Diferença
Trafegados (RQ2)	(~182,04 MB)	(~1,62 MB)	112x mais banda
Total de Linhas Úteis Inseridas (Dataset)	1.767 Pull Requests	3.210 Pull Requests	GraphQL extraiu 81% mais dados

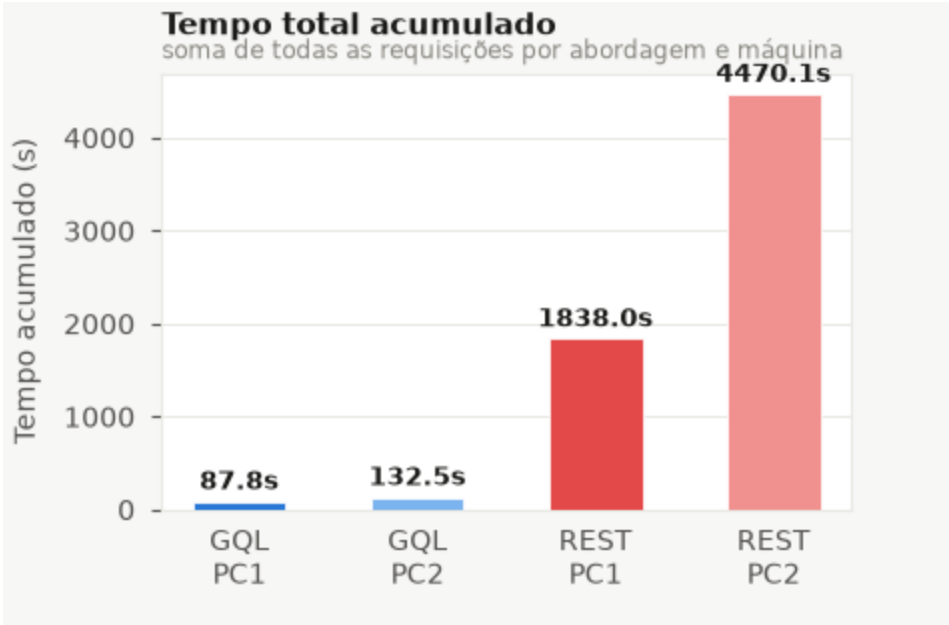


Figura 1: Comparativo de Tempo de Execução (REST v3 vs GraphQL v4)

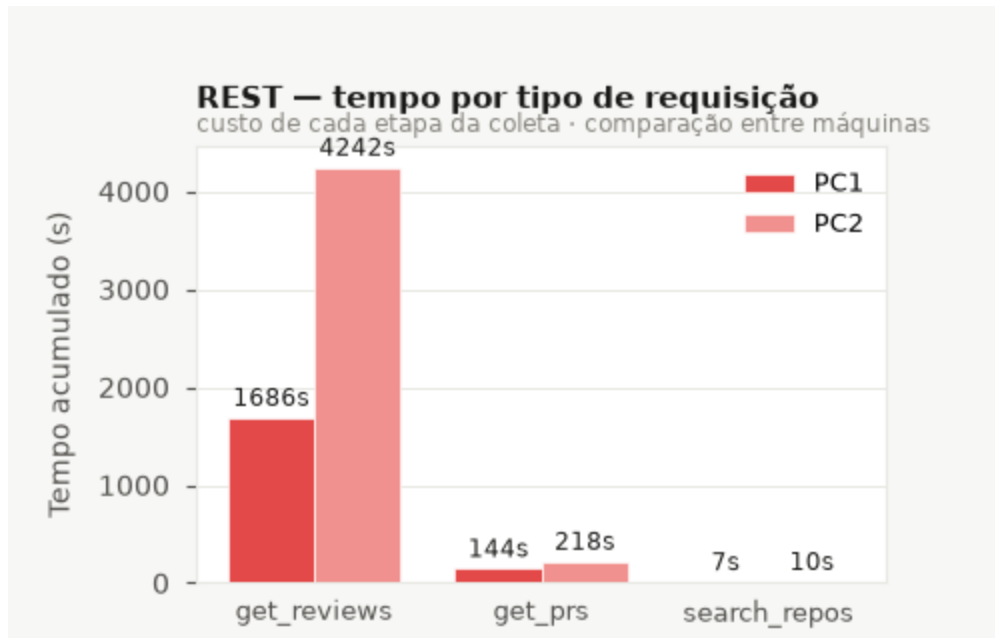


Figura 2: Evolução do Tempo de Execução acumulado por tipo de request



Figura 3: Comparativo de Tempo de Execução por tecnologia utilizada (REST v3 vs GraphQL v4)

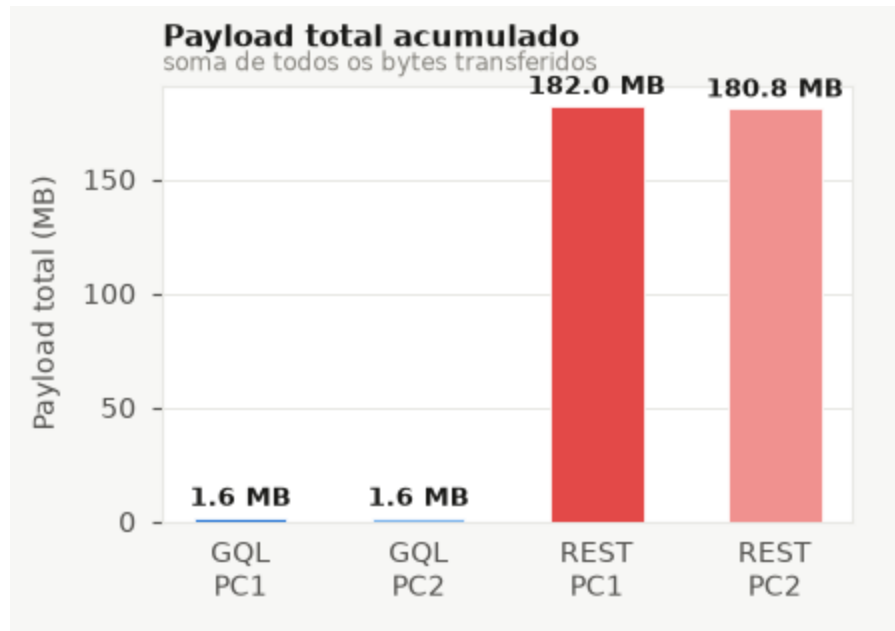


Figura 4: Evolução do Tamanho de Payload por Amostra.

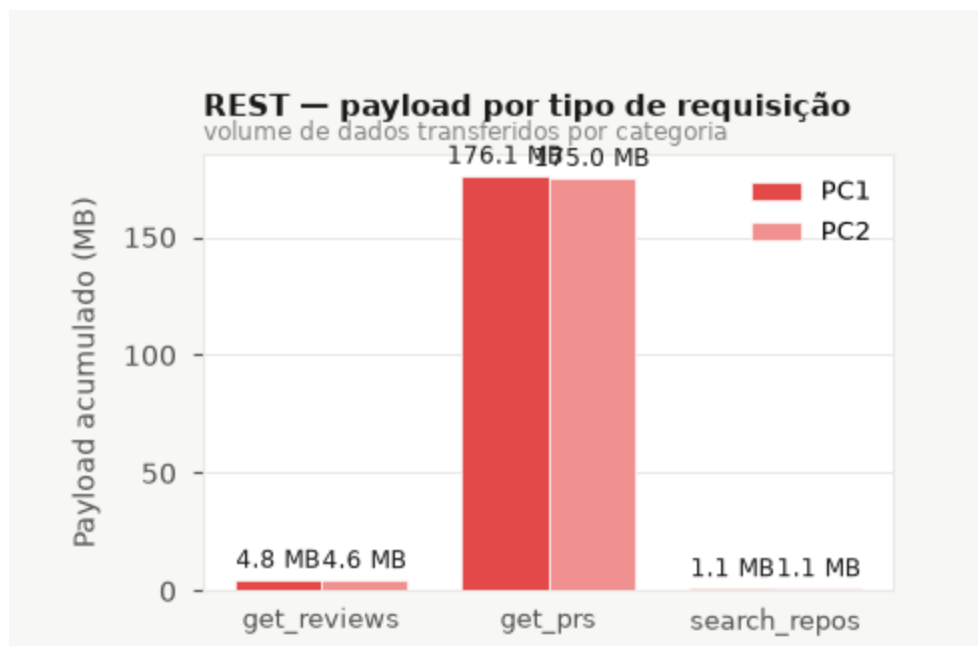


Figura 5: Relação de Volume de dados por tipo de request (REST v3 vs GraphQL v4)



Figura 6: Comparação final entre tamanho dos payloads e dados coletados (Pull Requests mapeados).

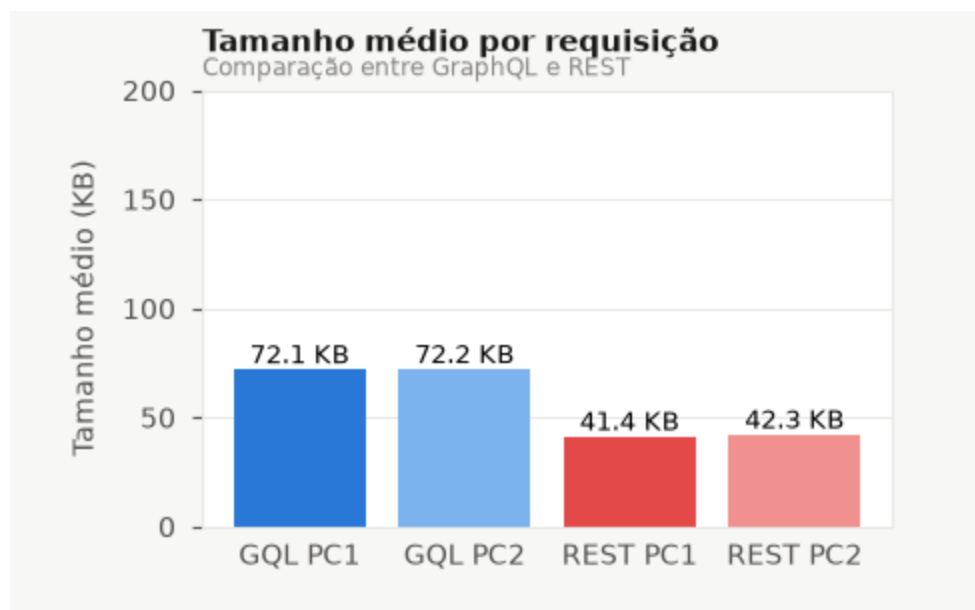


Figura 7: Tamanho Médio por Requisição (REST v3 vs GraphQL v4)

Análise da Pergunta de Pesquisa 1 (RQ1 - Tempo de Resposta)

Conforme ilustrado na Figura 1, Figura 2 e Figura 7, o tempo acumulado é drasticamente menor com GraphQL.

Os resultados coletados trazem evidências empíricas conclusivas para a rejeição da hipótese nula (H_0). Enquanto a API GraphQL levou apenas 1,46 minutos para varrer a amostragem de dados e preencher o conjunto final de registros, a rotina REST necessitou de 30,63 minutos de processamento ativo na mesma máquina sob as mesmas condições de rede.

O tempo médio gasto individualmente por cada uma das 22 requisições profundas do GraphQL

foi de aproximadamente 3,99 segundos. Em contrapartida, as requisições atômicas do REST foram individualmente mais velozes (média aproximada de 0,42 segundos por chamada). No entanto, o custo operacional acumulado gerado pelas 4.292 transações necessárias para mitigar o *under-fetching* anulou qualquer vantagem de velocidade isolada do REST, gerando um atraso catastrófico de escala global na rotina.

Análise da Pergunta de Pesquisa 2 (RQ2 - Tamanho do Payload)

Os dados de volume de tráfego, detalhados na Figura 3, Figura 4 e Figura 7, confirmam a ineficiência do modelo REST.

A análise volumétrica também confirmou a hipótese alternativa (H1) com larga margem de isolamento estatístico. A rotina REST forçou a transferência de 182,04 Megabytes através do canal de rede, enquanto a rotina GraphQL realizou a mesma entrega extraindo apenas 1,62 Megabytes. Essa diferença de magnitude ocorre devido ao excesso de dados irrelevantes trafegados pela estrutura de endpoints fixos do REST (*over-fetching*). Ao listar os Pull Requests, o REST trafega obrigatoriamente objetos completos contendo dados cadastrais de usuários, links de links internos, configurações de nós de árvore de commits e estruturas de metadados redundantes que não possuíam qualquer utilidade para o cálculo de Code Review delineado no experimento.

4. Discussão Final

Um aspecto crucial do experimento diz respeito à integridade dos dados extraídos. Embora o objetivo computacional de ambos os códigos fosse mapear o mesmo volume de dados, o arquivo `dataset_prs_graphql.csv` consolidou 3.210 Pull Requests válidos distribuídos em 96 repositórios distintos. Já o arquivo `dataset_prs_rest.csv` gerou apenas 1.767 linhas válidas mapeadas em 90 repositórios. Houve uma perda severa de dados úteis na extração via REST. A explicação para essa anomalia reside na vulnerabilidade da arquitetura a falhas de infraestrutura sob alta carga. Ao disparar mais de 4.200 chamadas de rede sequenciais, o script REST causou o esgotamento intermitente de recursos e sofreu com bloqueios preventivos de conexões e pequenas quedas de pacotes devido a picos de requisições. O código tratou essas quedas pulando os blocos falhos para evitar a interrupção abrupta do sistema, sacrificando registros inteiros (como os dados de repositórios de alta densidade de PRs, a exemplo de supabase/supabase e microsoft/terminal).

O GraphQL, operando com filtragem declarativa nativa diretamente nos servidores de banco de dados do GitHub, removeu o ruído antes do envio pelo cabo de rede. Ele reduziu drasticamente o estresse de conexão e garantiu uma captura íntegra, limpa e veloz dos dados de pesquisa. O experimento comprova empiricamente que o GraphQL resolve os problemas de desempenho e preserva a fidelidade das informações extraídas, mitigando as limitações técnicas impostas

pelo encadeamento assíncrono de APIs REST.